

Autonomous Systems

Decision Making for Mobile Robot Navigation using AlphaBot2

Group 18

Presentation 3

David Gaspar - 106541

Francisco Valério - 106533

Miguel Carneiro - 106218

Pedro Mónico - 106626

**MICRO SIMULATOR
UPDATES**

MDP Algorithm emphasizing
Performance Results

CAMERA CALIBRATION

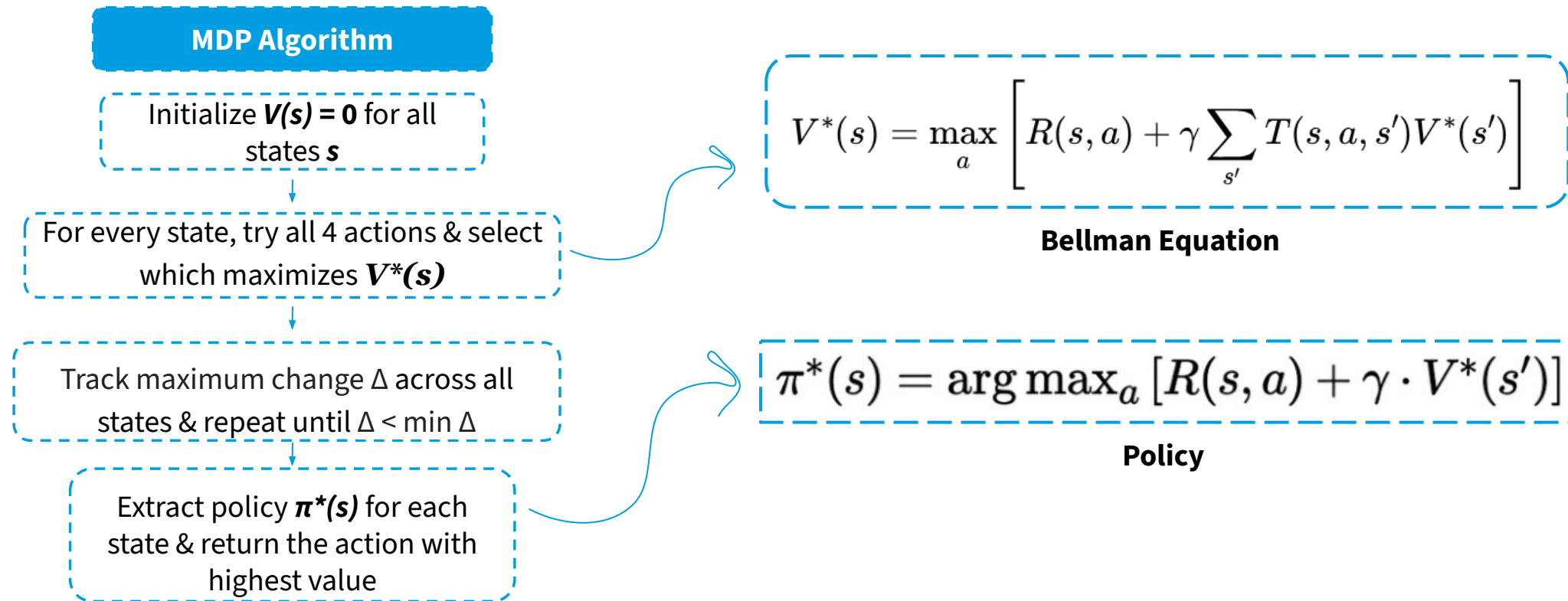
Calibration Methods
ARuco Markers Detection

LINE SENSORS

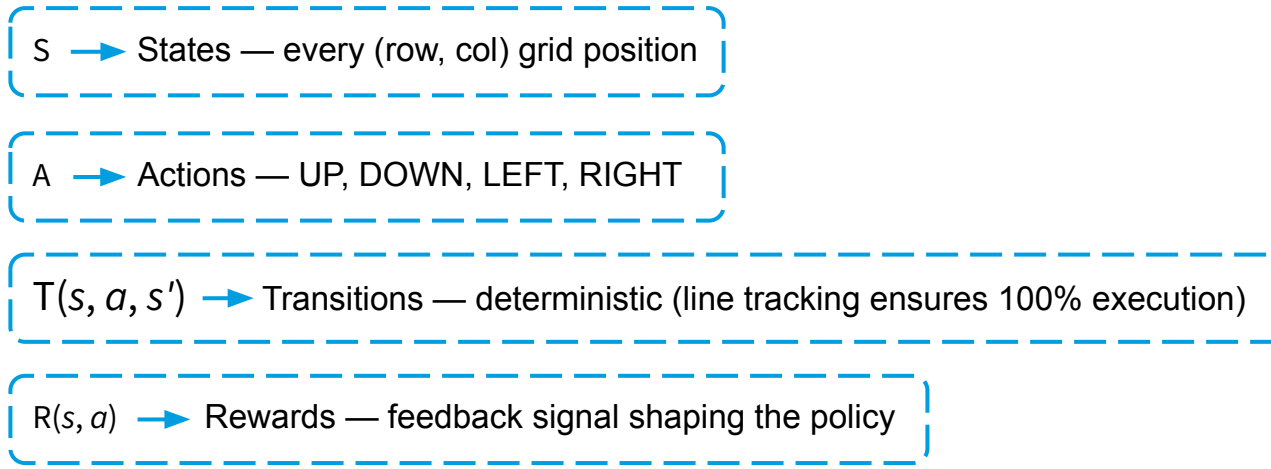
Why use them?
Hardware & Sensor Data
Tests Outputs & Results
Control Implementation

NEXT STEPS OF WORKPLAN

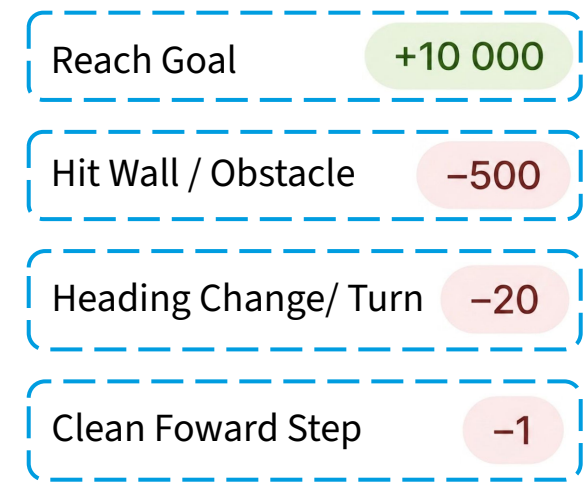
MICRO SIMULATOR UPDATES



MDP Parameters →



Reward Definition →



Why deterministic? **Line - tracking sensors and algorithms lock the robot onto the grid lines**, so the command action is achieved with 100% reliability. The stochastic drift model is no longer needed.

```

29 def main():
63     print(f"State: {state} | Reward: {reward}")
64     print_grid(env, policy=policy, robot_pos=state)
65
66     if done:
67         print(f"✓ Goal reached in {steps} steps!")
    
```

PROBLEMS OUTPUT DEBUG CONSOLE **TERMINAL** PORTS

(solvable map found on attempt 1)

=== INITIAL GRID ===

```

+---+---+---+---+
| S | . | . | . |
+---+---+---+---+
| . | . | . | . |
+---+---+---+---+
| . | . | . | . |
+---+---+---+---+
| . | . | . | . |
+---+---+---+---+
| . | . | . | . |
+---+---+---+---+
| . | . | . | G |
+---+---+---+---+
    
```

Value iteration converged in 82 iterations ($\Delta=9.59e-04$)

=== OPTIMAL POLICY ===

```

+---+---+---+---+
| → | ↓ | ↓ | ← |
+---+---+---+---+
| → | ↓ | ↓ | ↓ |
+---+---+---+---+
| ↑ | ↓ | ↓ | ↓ |
+---+---+---+---+
| → | → | → | ↓ |
+---+---+---+---+
| → | → | → | ★ |
+---+---+---+---+
    
```

=== ROBOT FOLLOWING POLICY ===

Step 1 | Action: RIGHT | State: (0, 1) | Reward: -20

CAMERA CALIBRATION

CALIBRATION



The calibration was done through a **ChArUco** board

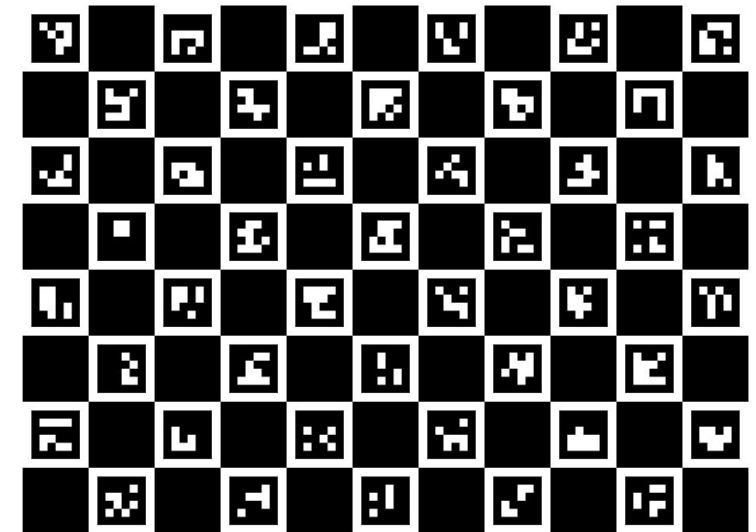
Calibration captures were guided by the ROS 2 **camera_calibration** tool, monitoring **X**, **Y**, **size** and **skew** coverage metrics

The output was a **.yaml file** containing:

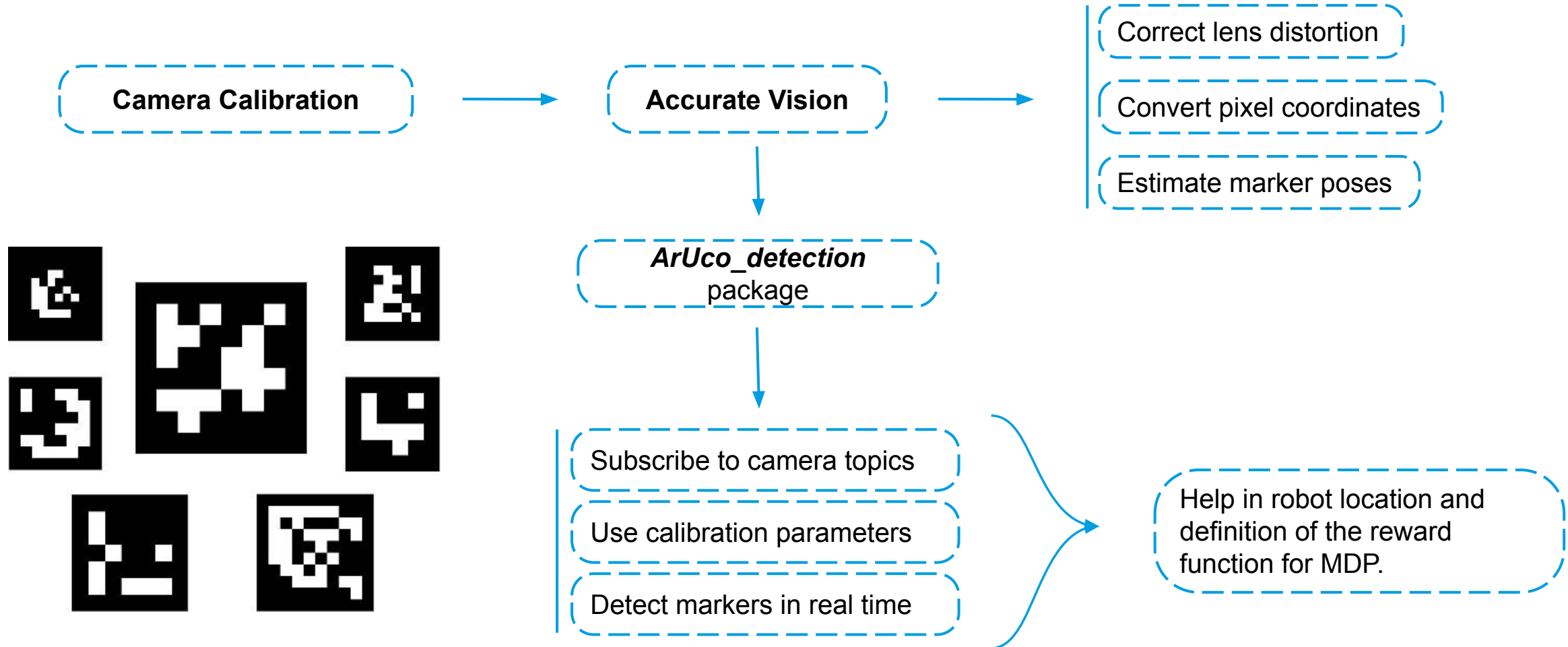
Intrinsic Camera Matrix

Projection Matrix

Distortion Model + Coefficients



ChArUco board

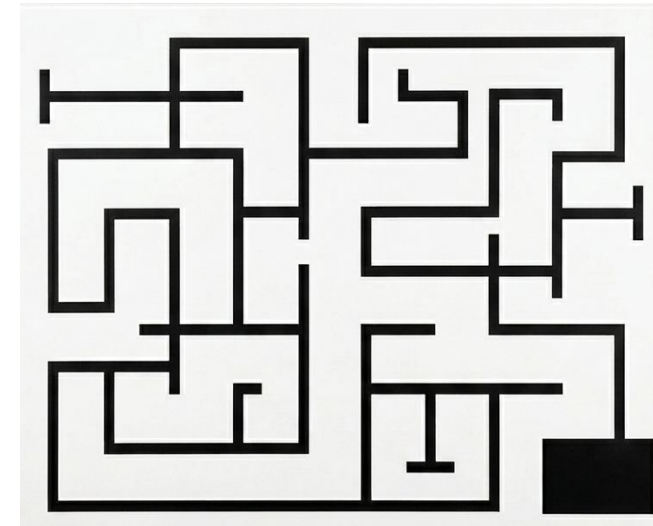


LINE SENSORS

Structured navigability: The line defines a fixed, predictable path the robot can always follow, making navigation deterministic and repeatable

Works within the robot's limitations: AlphaBot2 has no odometry or LiDAR, so line sensors are the practical choice

Real-time response: Sensors react instantly, making the robot fast to correct its path



Example of Maze Sketch

5x ITR20001/T — reflective infrared photoelectric sensors positioned on the underside of the AlphaBot2

Each sensor **emits Infrared Light** downward and reads how much is reflected back

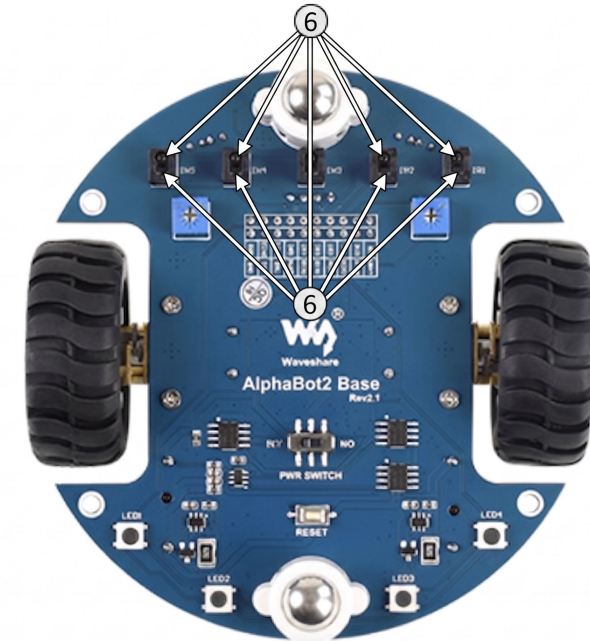
→ **Dark surface (line)** → low reflection → low value

→ **Light surface (floor)** → high reflection → high value

Sensor outputs a value
between **0** and **1023**

→ ~**0** → sensor is directly over the line (black)

→ ~**1023** → sensor is over the floor (white)



AlphaBot2 Hardware

Experiment

Each sensor was passed over the black line one at a time and then to white again

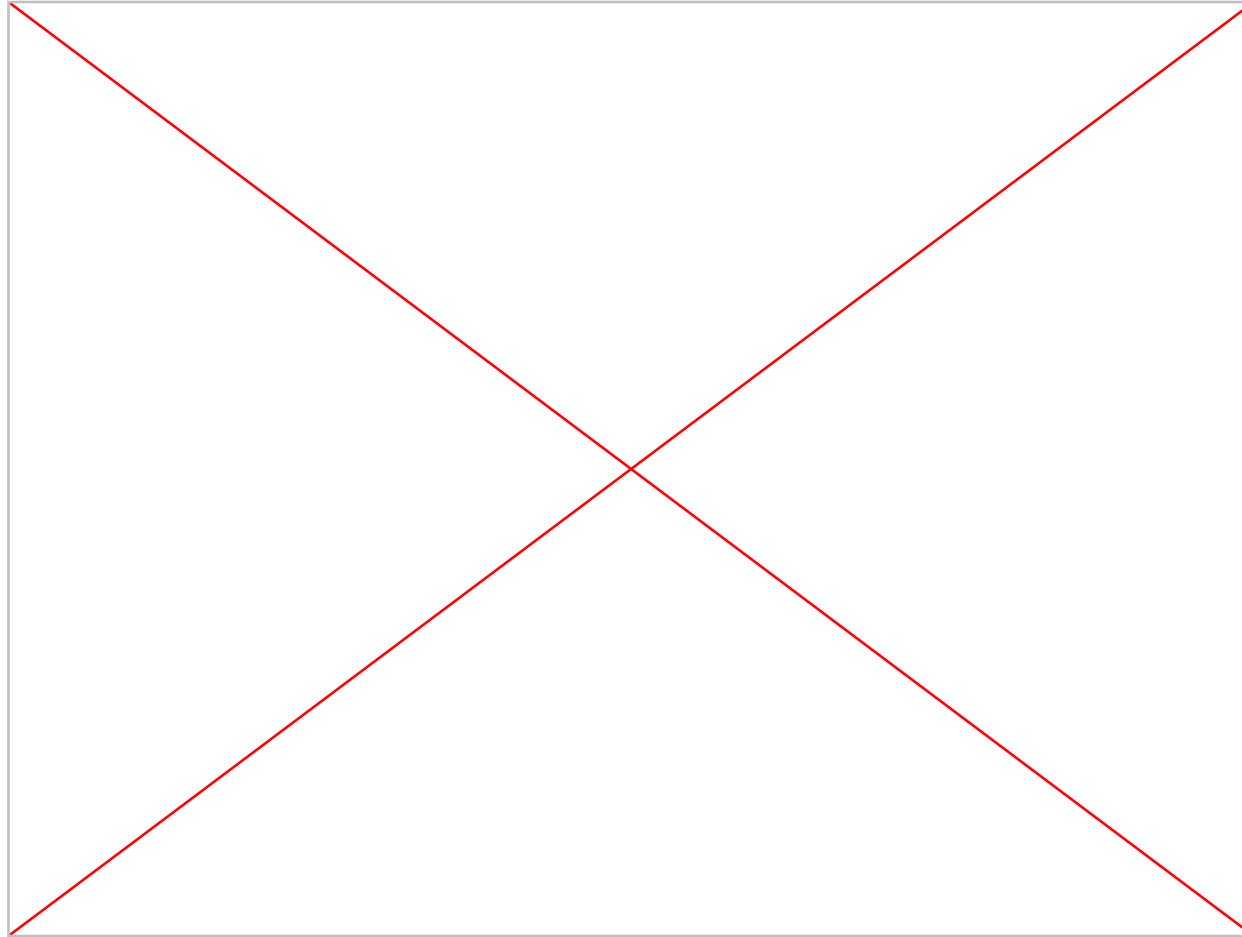
All 5 sensors were set over the black line at the same instant

Other colors were tested to measure their reflectance but there was not one that would sit in the stable range between [400,900]

Solution for obstacle detection is to **use the vertical sensors**

Black print values sit in the range [200, 400]
White print values sit in the range [960, 980]





NEXT STEPS OF WORKPLAN

TASK	DESCRIPTION
Control Logic for Intersections & Obstacles	Implement decision-making when the robot encounters an intersection or an obstacle on the line
Full Run Experiment	Test a complete maze run using the current knowledge of the track layout
ArUco Marker Detection & Identification	Integrate camera-based detection of ArUco markers to determine the robot's position within the maze

Thank You!